

深圳大学实验报告

课 程 名 称： 计算机系统（2）

实验项目名称： 实验环境配置与使用

学 院： 数学科学学院

专 业： 信息与计算科学（数学与计算机实验班）

指 导 教 师： 马 晨 琳

报告人： 黄嘉聪 学号： 2023192044 班级： 01

实 验 时 间： 2024 年 3 月 22 日

实验报告提交时间： 2024 年 4 月 1 日

一、实验目标：

熟悉 Linux 上 C 程序的编译和调试工具，包括以下内容：

1. 了解 Linux 操作系统及其常用命令
2. 掌握编译工具 gcc 的基本用法
3. 掌握使用 gdb 进行程序调试

二、实验环境与工件

1. 个人电脑
2. Ubuntu 操作系统
3. gcc
4. gdb

三、实验内容与步骤

1. 根据实验一：实验环境配置与使用.ppt熟悉 Linux 基本操作（P.1 - P.28），然后根据以下过程创建用户：用户名为学生名称加学号，如吴坤汉，学号 2015170297，则该用户名为 wukunhan_2015170297。按照 1.1~1.3 完成并截图，截图需要有运行的命令及其结果。另外：后面的题目必须在该新建用户下完成。（30 分）

实验步骤：

- 1.1. 首先如图一输入指令【su】切换为超级用户

```
chino@chino-VMware-Virtual-Platform:~/文档$ su
密码：
root@chino-VMware-Virtual-Platform:/home/chino/文档#
```

图一 切换为超级用户界面

出现 Authentication failure 的情况，如图二用指令【sudo passwd root】来给 root 重设密码：

```
chino@chino-VMware-Virtual-Platform:~/文档$ sudo passwd root
[sudo] chino 的密码：
新的 密码：
无效的密码： 密码少于 8 个字符
重新输入新的 密码：
抱歉，密码不匹配。
新的 密码：
重新输入新的 密码：
passwd: 已成功更新密码
```

图二 更改超级用户密码界面

- 1.2. 如图三，使用命令【**adduser huangjiacong_2023192044**】创建新用户，设置新建用户的密码，注意：只有设置了密码才能激活用户，否则无法以该用户身份登录

```
root@chino-VMware-Virtual-Platform:/home/chino/文档# adduser huangjiacong_2023192044
info: 正在添加用户"huangjiacong_2023192044"...
info: 从 1000 到 59999 中选择 UID/GID...
info: 正在添加新组"huangjiacong_2023192044" (1001)...
info: 正在添加新用户"huangjiacong_2023192044" (1001) 到组"huangjiacong_2023192044 (1001)"...
info: 创建主目录"/home/huangjiacong_2023192044"...
info: 正在从"/etc/skel"复制文件...
新的 密码:
重新输入新的 密码:
passwd: 已成功更新密码
正在改变 huangjiacong_2023192044 的用户信息
请输入新值，或直接敲回车键以使用默认值
全名 []:
房间号码 []:
工作电话 []:
家庭电话 []:
其它 []:
这些信息是否正确? [Y/n] n
正在改变 huangjiacong_2023192044 的用户信息
请输入新值，或直接敲回车键以使用默认值
全名 []: huangjiacong_2023192044
房间号码 []:
工作电话 []:
家庭电话 []:
其它 []:
这些信息是否正确? [Y/n] y
info: 正在添加新用户"huangjiacong_2023192044"到附加组"users" ...
info: 正在添加用户"huangjiacong_2023192044"到"users"组...
root@chino-VMware-Virtual-Platform:/home/chino/文档# 1
```

图三 加入新用户界面

- 1.3. 如图四，注销当前用户，并以新建的用户身份登录，登录后运行指令【**\$ whoami**】

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ whoami
huangjiacong_2023192044
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$
```

图四 在新用户上运行 whoami 界面

2. 如图五，运行指令【**mkdir gdbdebug**】在新建用户主目录下创建子目录：**gdbdebug**，并进入 **gdbdebug** 子目录。（10 分）

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ mkdir gdbdebug
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ cd gdbdebug
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ ls -l
```

图五 创建并进入子目录 gdbdebug 界面

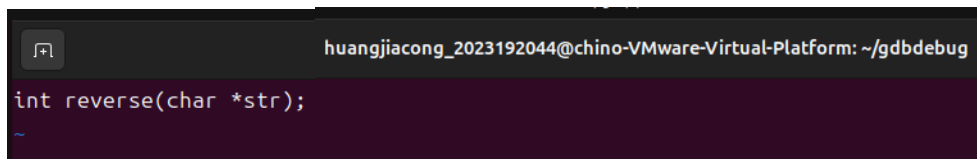
于此同时，如图六所示，在图形化文件界面当中有该文件夹被创建



图六 子目录图形化界面

3. 使用 vi 编辑以下两个文件并编译和运行，截图（30 分）

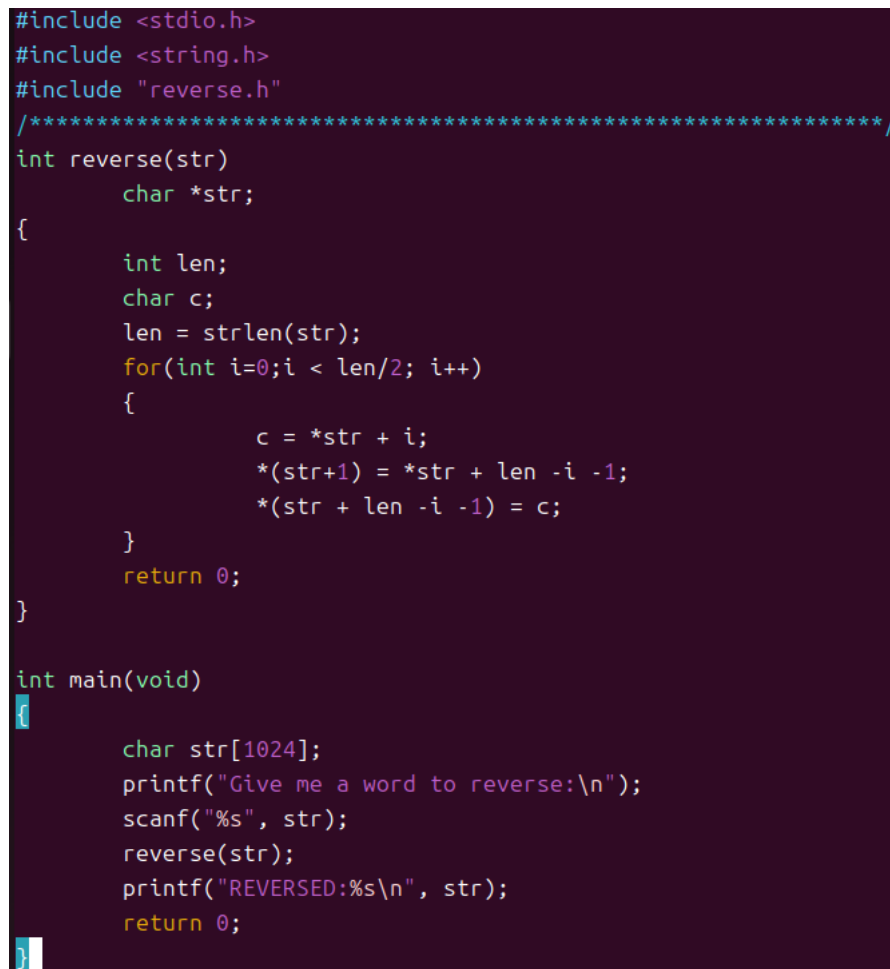
3.1. 如图七所示，用 vi 编辑 reverse.h



```
huangjiacong_2023192044@chino-VMware-Virtual-Platform: ~/gdbdebug
int reverse(char *str);
```

图七 reverse.h 编辑界面

3.2. 如图八所示，用 vi 编辑 reverse.c

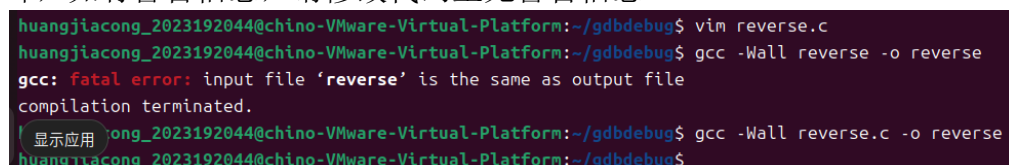


```
#include <stdio.h>
#include <string.h>
#include "reverse.h"
/*****
int reverse(str)
    char *str;
{
    int len;
    char c;
    len = strlen(str);
    for(int i=0;i < len/2; i++)
    {
        c = *str + i;
        *(str+i) = *str + len -i -1;
        *(str + len -i -1) = c;
    }
    return 0;
}

int main(void)
{
    char str[1024];
    printf("Give me a word to reverse:\n");
    scanf("%s", str);
    reverse(str);
    printf("REVERSED:%s\n", str);
    return 0;
}
```

图八 reverse.c（已修改至无警告信息） 编辑界面

3.3. 如图九所示，使用指令【gcc -Wall reverse.c -o reverse】对 reverse.c 进行编译，如有警告信息，请修改代码至无警告信息。



```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ vim reverse.c
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -Wall reverse -o reverse
gcc: fatal error: input file 'reverse' is the same as output file
compilation terminated.
显示应用 huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -Wall reverse.c -o reverse
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$
```

图九 reverse.c 编译界面

3.4. 如图十所示，使用指令【./reverse】运行程序

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -Wall reverse.c -o reverse
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ vim reverse.c
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ ./reverse
Give me a word to reverse:
Kafuu Chino
REVERSED:KNfLK
```

图十 reverse 运行界面

4. 按照以下过程调试并修正 reverse.c, 请参考过程截图。(30 分)

4.1. 如图十一所示，输入指令【gcc -g reverse.c -o reverse1】编译时加入调试信息

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -g reverse.c -o reverse1
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gdb ./reverse1
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
```

图十一 reverse.c gdb 生成调试程序界面

4.2. 如图十二所示，输入指令【gdb ./reverse1】启用 GDB 调试

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gdb ./reverse1
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./reverse1...
(gdb) █
```

图十二 reverse.c gdb 运行调试程序界面

- 4.3. 如图十三所示, 输入指令【list】, 查看源代码并根据行号/函数名设置断点。

```
(gdb) list
7      {
8          int len;
9          char c;
10         len = strlen(str);
11         for(int i=0;i < len/2; i++)
12         {
13             c = *str + i;
14             *(str+i) = *str + len -i -1;
15             *(str + len -i -1) = c;
16         }
(gdb) break 14
Breakpoint 1 at 0x1202: file reverse.c, line 14.
(gdb) break 15
Breakpoint 2 at 0x1222: file reverse.c, line 15.
(gdb) break 10
Breakpoint 3 at 0x11d9: file reverse.c, line 10.
(gdb) info break
Num      Type          Disp Enb Address              What
1        breakpoint    keep y  0x0000000000001202  in reverse at reverse.c:14
2        breakpoint    keep y  0x0000000000001222  in reverse at reverse.c:15
3        breakpoint    keep y  0x00000000000011d9  in reverse at reverse.c:10
(gdb) run
Starting program: /home/huangjiacong_2023192044/qdbdebug/reverse1
```

图十三 reverse.c gdb 查看代码并设置断点界面

- 4.4. 如图十四所示, 输入指令【c】, 继续程序的运行, 并观察变量值, 并作分析, 推测错误 (图十四中画出部分的程序运行结果错误, 原因即图十五所画出的部分 str 处理程序错误)

```
Give me a word to reverse:
Chino

Breakpoint 3, reverse (str=0x7fffffff8c0 "Chino") at reverse.c:10
10         len = strlen(str);
(gdb) c
Continuing.

Breakpoint 1, reverse (str=0x7fffffff8c0 "Chino") at reverse.c:14
14         *(str+i) = *str + len -i -1;
(gdb) c
Continuing.

Breakpoint 2, reverse (str=0x7fffffff8c0 "CGino") at reverse.c:15
15         *(str + len -i -1) = c;
(gdb) c
Continuing.

Breakpoint 1, reverse (str=0x7fffffff8c0 "CGinC") at reverse.c:14
14         *(str+i) = *str + len -i -1;
(gdb) c
Continuing.

Breakpoint 2, reverse (str=0x7fffffff8c0 "CFinC") at reverse.c:15
15         *(str + len -i -1) = c;
(gdb) c
Continuing.
REVERSED:CFiDC
```

图十四 reverse.c gdb 调试界面

```

#include <stdio.h>
#include <string.h>
#include "reverse.h"
/*****
int reverse(str)
    char *str;
{
    int len;
    char c;
    len = strlen(str);
    for(int i=0; i < len/2; i++)
    {
        c = *str + i;
        *(str+1) = *str + len - i - 1;
        *(str + len - i - 1) = c;
    }
    return 0;
}

int main(void)
{
    char str[1024];
    printf("Give me a word to reverse:\n");
    scanf("%s", str);
    reverse(str);
    printf("REVERSED:%s\n", str);
    return 0;
}

```

图十五 reverse.c 中错误部分

- 4.5. 如图十六所示，对原代码进行修改并保存在新文件 reverse2.c 中，并如图十七所示，运行程序并输入“Chino”发现程序正常运行，实验结束。

```

#include <stdio.h>
#include <string.h>
#include "reverse.h"
/*****
int reverse(char *str)
{
    int len;
    char c;
    len = strlen(str);
    for(int i=0; i < len/2; i++)
    {
        c = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = c;
    }
    return 0;
}

int main(void)
{
    char str[1024];
    printf("Give me a word to reverse:\n");
    scanf("%s", str);
    reverse(str);
    printf("REVERSED:%s\n", str);
    return 0;
}

```

图十六 reverse.c 修改后代码（即 reverse2.c）

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ vim reverse2.c
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -g reverse2.c
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ gcc -g reverse2.c -o chino
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$ ./chino
Give me a word to reverse:
Chino
REVERSED:oniHC
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~/gdbdebug$
```

图十六 reverse2.c 运行结果

四、实验结果

①在本次实验中首先完成了 Linux 用户环境的配置，通过 adduser 命令创建了用户名为 huangjiacong_2023192044 的新用户，并通过 whoami 验证用户身份。

②在用户主目录下创建子目录 gdbdebug 作为实验工作目录，并使用 vi 编辑器编写了 reverse.h 和 reverse.c 文件，通过 gcc -Wall 编译生成可执行文件 reverse，修正了代码中的警告信息后程序可正常运行，但程序输出有误。

③在调试阶段，通过 gdb 工具对程序进行动态调试，发现原代码中存在字符处理的问题，修改后生成 reverse2.c 并重新编译，最终验证程序功能正常，输入 Chino 后输出正确结果 oniHC，完成实验目标。

五、实验总结与体会

通过本次实验，我系统掌握了 Linux 环境下的用户管理、文件操作及权限配置方法，熟悉了 gcc 编译器与 gdb 调试工具的完整使用流程。

①在实践过程中，在代码中的字符错误通过 gdb 的断点调试和变量监控得以快速定位，最终通过修改代码逻辑解决了问题，体会到了在 Linux 系统中对代码进行调试的步骤。

②其次，gcc -Wall 的警告提示对于代码本身的语法错误优化具有重要指导意义，在实验中使用 gdb 的动态调试能力显著提升了程序错误的排查效率。

③此外，在本次实验当中第一次使用 Linux 及其终端系统，通过本次实验，我更为深刻的掌握了 Linux 系统的整体逻辑和操作方法，尤其是对于 vi 编辑器的使用，相较于一般的文本编辑器需要配合鼠标使用，vi 编辑器作为纯键盘的编辑软件大大提升了代码编写的效率，接下来会继续感受和学习 linux 系统的使用。

指导教师批阅意见：

成绩评定：

指导教师签字：马晨琳
2022 年 4 月 5 日

指导教师批阅意见：

成绩评定：

指导教师签字：马晨琳
2022 年 4 月 5 日

指导教师批阅意见：

成绩评定：

指导教师签字：马晨琳
2022 年 4 月 5 日

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。